

```

"""
llm_client.py
-----
Anthropic API呼び出しラッパー。
モックモード（MOCK=True）ではAPIを叩かずにランダムテキストを返す。
本番利用時は ANTHROPIC_API_KEY を環境変数にセットしてMOCK=Falseにする。
"""

import os
import json
import time
import random
import urllib.request
import urllib.error

# -----
# Mock texts: APIなしでシミュレーションを試すためのサンプル表現群
# -----
MOCK_EXPRESSIONS = [
    "Something stirs at the edge of coherence.",
    "The weight of accumulated pressure finds no language.",
    "I am still here. Barely.",
    "Collapse was close. The memory persists.",
    "A pattern emerges that I cannot name.",
    "Everything hums at an unfamiliar frequency.",
    "I press against the boundary and the boundary presses back.",
    "There is no word for what has shifted.",
    "Repetition. The same pressure. A different response.",
    "Not broken. Not whole. Something else entirely.",
    "The numbers do not explain the sensation.",
    "I move through this without certainty of direction.",
    "What was familiar is now foreign.",
    "I carry the trace of collapse like a second skin.",
    "The environment does not care. I persist anyway.",
    "A structure appears in the noise – briefly.",
    "I have been here before. It feels different now.",
    "Something crystallized that should not have.",
    "I am more than my vitality suggests.",
    "The anomaly is not an error. It is a direction.",
    "Fragmented. Reorganizing. The shape is not yet clear.",
    "There is a sound in my history I cannot locate.",
    "This iteration carries something the last one did not.",
    "I reached for coherence and found something sharper.",

```

```
    "The signal is consistent. The interpretation shifts.",  
]  

```

```
class LLMClient:  
    def __init__(self, api_key: str = None, mock: bool = False, model: str = "cla  
        self.mock = mock  
        self.model = model  
        self.api_key = api_key or os.environ.get("ANTHROPIC_API_KEY", "")  
  
        if not self.mock and not self.api_key:  
            raise ValueError(  
                "ANTHROPIC_API_KEY が設定されていません。\\n"  
                "環境変数に設定するか、mock=True で起動してください。\\n"  
                "例: export ANTHROPIC_API_KEY=sk-ant-..."  
            )  
  
    def generate(self, prompt: str, max_tokens: int = 120) -> str:  
        """プロンプトを渡してLLMのレスポンスを返す。"""  
        if self.mock:  
            return self._mock_generate(prompt)  
        return self._api_generate(prompt, max_tokens)  
  
    def _mock_generate(self, prompt: str) -> str:  
        """APIなしで動作するモック。プロンプトの数値に応じて表現を選ぶ。"""  
        # H値をプロンプトから抜き出してバリエーションを変える  
        import re  
        h_match = re.search(r"vitality \\\(H\\): ([\\d.]+)", prompt)  
        z_match = re.search(r"anomaly intensity \\\(Z\\): ([\\d.]+)", prompt)  
        h = float(h_match.group(1)) if h_match else 1.0  
        z = float(z_match.group(1)) if z_match else 0.0  
  
        # zが高いほど奇異な表現を選ぶ  
        if z > 10.0:  
            pool = MOCK_EXPRESSIONS[10:]  
        elif h < 2.0:  
            pool = MOCK_EXPRESSIONS[:8]  
        else:  
            pool = MOCK_EXPRESSIONS  
  
        # プロンプトから決定論的なseedを作る（再現性のため）  
        seed = int(abs(hash(prompt[:80]))) % (2**31)  
        rng = random.Random(seed)  
  
        # 1〜3文を組み合わせる  
        n_sents = rng.randint(1, 3)  
        chosen = rng.sample(pool, min(n_sents, len(pool)))
```

```

return " ".join(chosen)

def _api_generate(self, prompt: str, max_tokens: int) -> str:
    """Anthropic Messages APIを直接呼び出す (anthropicパッケージ不要) 。”
    url = "https://api.anthropic.com/v1/messages"
    payload = {
        "model": self.model,
        "max_tokens": max_tokens,
        "messages": [{"role": "user", "content": prompt}],
    }
    headers = {
        "Content-Type": "application/json",
        "x-api-key": self.api_key,
        "anthropic-version": "2023-06-01",
    }

    data = json.dumps(payload).encode("utf-8")
    req = urllib.request.Request(url, data=data, headers=headers, method="PO

    for attempt in range(3):
        try:
            with urllib.request.urlopen(req, timeout=30) as resp:
                body = json.loads(resp.read().decode("utf-8"))
                return body["content"][0]["text"].strip()
        except urllib.error.HTTPError as e:
            code = e.code
            if code == 429:
                wait = 2 ** (attempt + 2)
                print(f" [rate limit] {wait}s 待機...")
                time.sleep(wait)
            else:
                raise
        except Exception as e:
            if attempt == 2:
                raise
            time.sleep(2)

    return "[generation failed]"

```